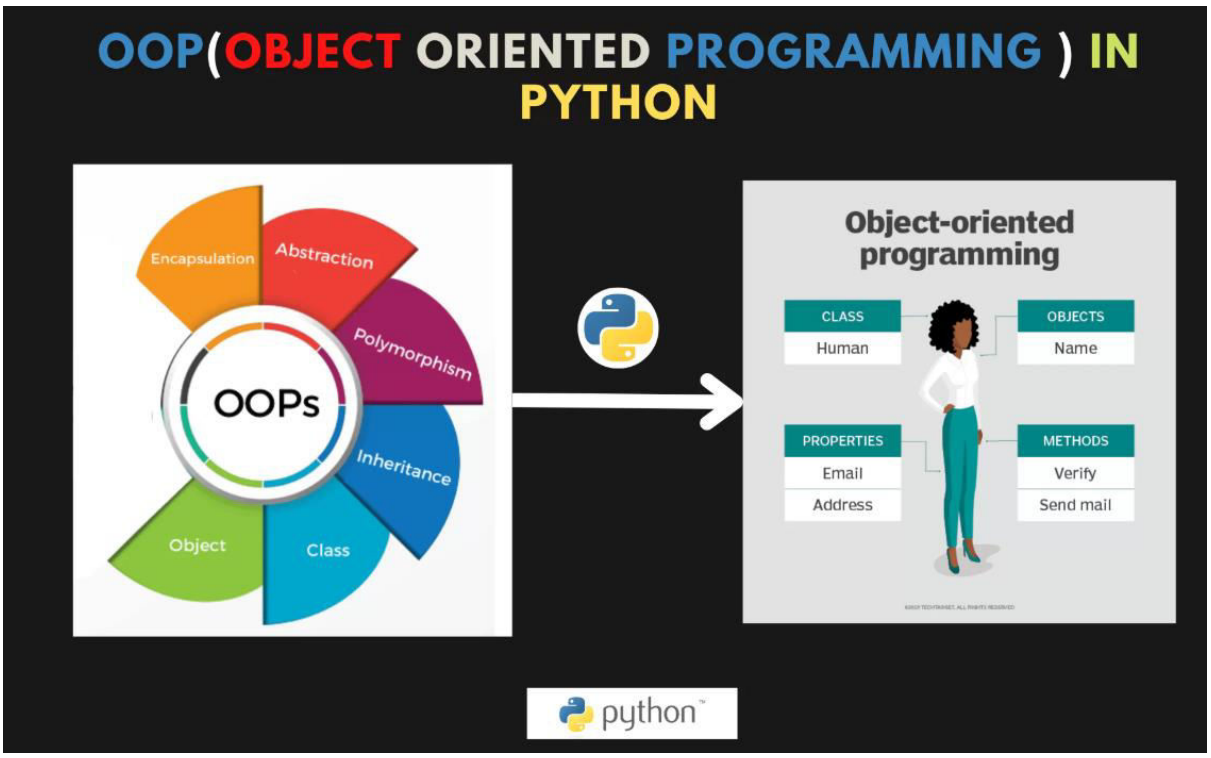
Object-Oriented Programming in Python

Introduction

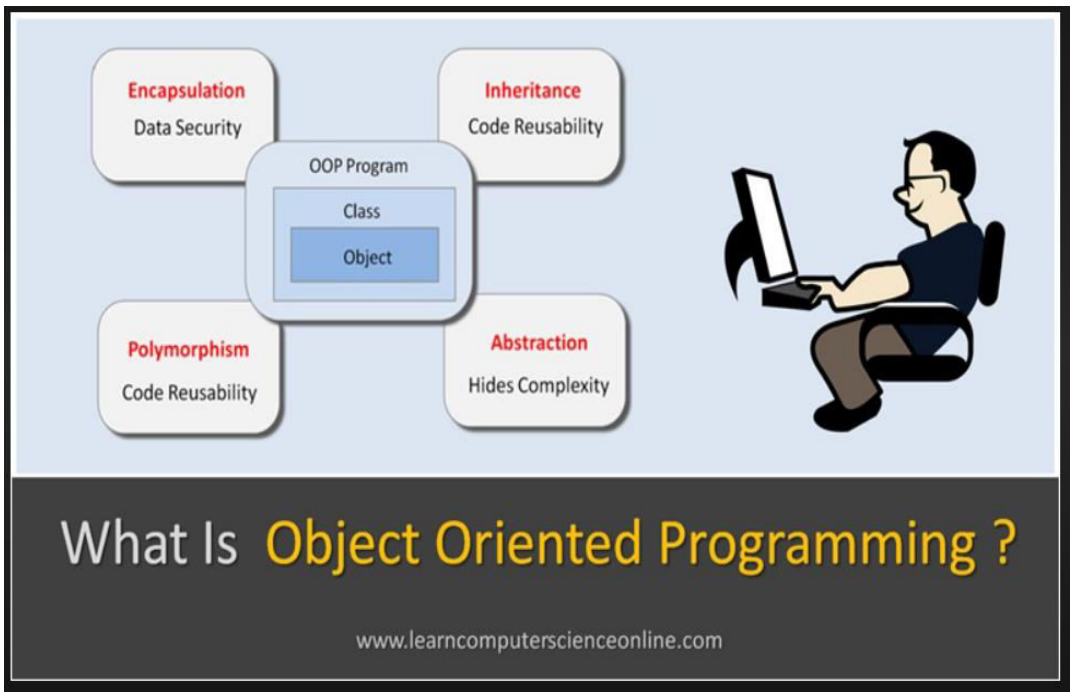
Object-Oriented Programming (OOP) is a programming paradigm that uses objects and classes to structure code efficiently. Python, being an object-oriented language, supports OOP features like classes, objects, inheritance, polymorphism, and encapsulation.



Why Use Object-Oriented Programming?

- **Code Reusability**: Classes and objects allow for modular code, reducing redundancy.
- **Encapsulation**: Restricts direct access to some of an object's components.

- **Inheritance**: Promotes code reusability by allowing new classes to derive properties from existing ones.
- **Polymorphism**: Enables a single function or method to work with different types of objects.
- **Scalability**: Helps in managing large codebases efficiently.



Advantages & Limitations

Advantages:

- Enhances readability and maintainability.
- Helps in real-world modeling (e.g., in game development, GUI applications).
- Facilitates debugging and testing.
- Encourages data hiding and abstraction.

Limitations:

- Requires more memory than procedural programming.
- May introduce complexity for small-scale programs.
- Execution speed is slower compared to procedural programming.

Key Concepts of OOP

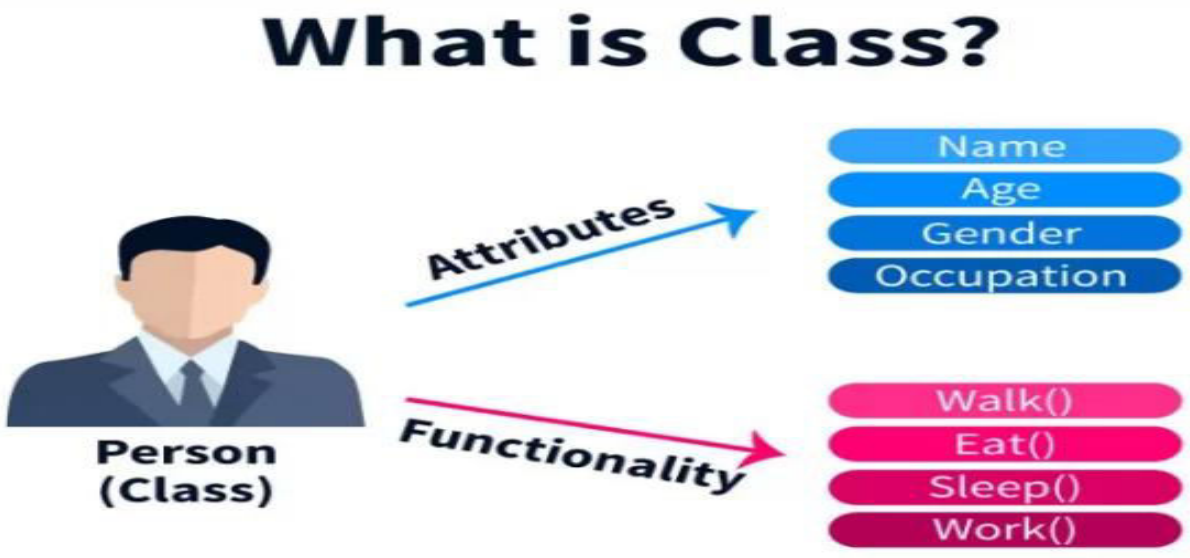
1. Class & Object in Python

Class:

A class in Python is a blueprint for creating objects. It defines a set of attributes (variables) and behaviors (methods) that the objects created from the class will have.

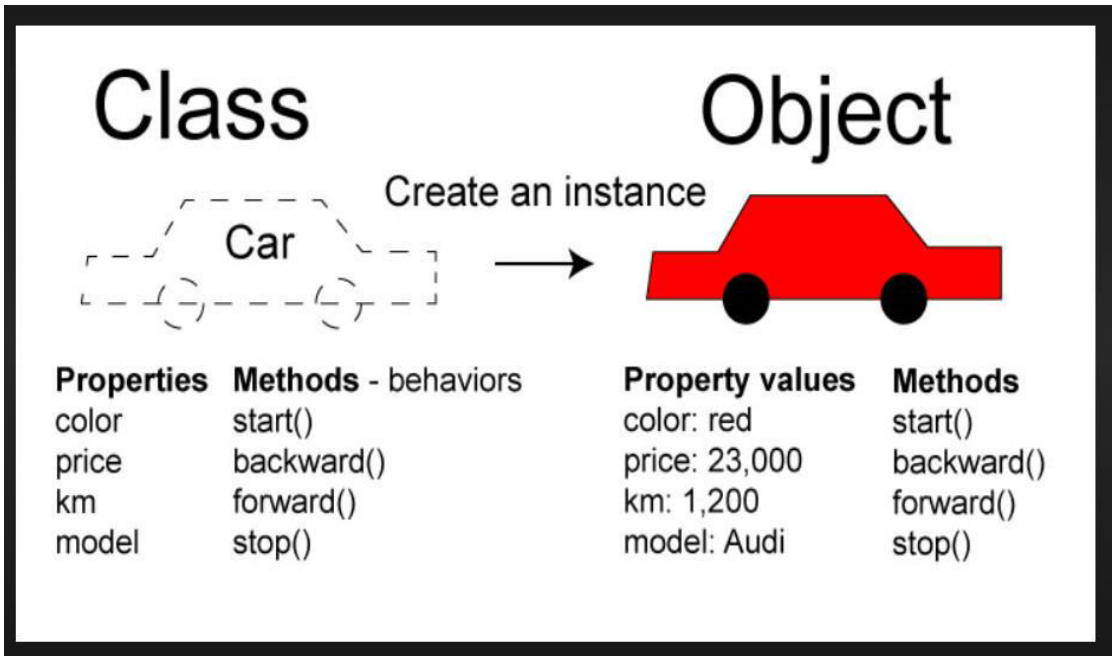
Object:

An object is an instance of a class. When a class is defined, no memory is allocated until an object of that class is created. Each object can have unique values for the attributes defined in the class.



Why Use Classes & Objects?

1. Code Reusability: Classes allow for reusable code by creating multiple objects from a single blueprint.
2. Encapsulation: Data hiding and restricting access to certain details enhance security.
3. Organization & Maintainability: Structuring code using classes makes it more modular and manageable.
4. Real-world Modeling: OOP allows developers to model real-world entities (e.g., a Car class with attributes like brand, model, speed).



Example of Class & Object in Python

python

```python
class Car:
    def __init__(self, brand, model):  # Constructor method
        self.brand = brand  # Instance variable
        self.model = model  # Instance variable

    def display(self):
        print(f"Car Brand: {self.brand}, Model: {self.model}")

# Creating an object (instance of Car class)
car1 = Car("Toyota", "Corolla")
car1.display()
```

Output:

yaml

```yaml
Car Brand: Toyota, Model: Corolla
```

OBJECT	CLASS
Object is an instance ofa class.	Class is a blue print from which objects are created
Object is a real world entity such as chair, pen. table, laptop etc.	Class is a group of similar objects.
Object is a physical entity.	Class is a logical entity.
Object is created many times as per requirement.	Class is declared once.
Object allocates memory when it is created.	Class doesn't allocated memory when it is created.
Object is created through new keyword. Employee ob = new Employee();	Class is declared using class keyword. class Employee{}
There are different ways to create object in java:- New keyword, newinstance() method, clone() method, And deserialization.	There is only one way to define a class, i.e., by using class keyword.

Inbuilt Class: Python has built-in classes like list, tuple, dict, etc.

Creating a User-defined Class:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

car1 = Car("Toyota", "Corolla")
print(car1.brand, car1.model)
```

Instance Variable & Class Variable

1. Instance Variable

- Instance variables are specific to an instance of a class.
- Defined inside a method using self.variable_name.
- Each instance of a class has its own copy of the instance variable.
- Changes to an instance variable do not affect other instances.

Example:

```python
class Student:
    def __init__(self, name, age):
        self.name = name  # Instance variable
        self.age = age    # Instance variable

# Creating instances
s1 = Student("Alice", 20)
s2 = Student("Bob", 22)

print(s1.name)  # Alice
print(s2.name)  # Bob
```

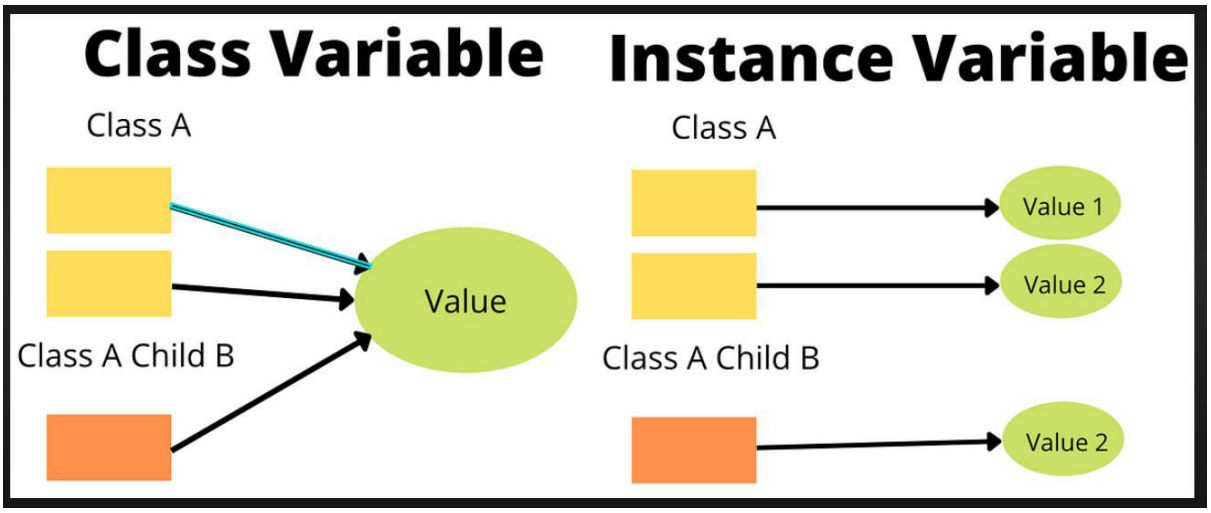
2. Class Variable

- Class variables are shared among all instances of a class.
- Defined outside of any method but inside the class.
- Accessed using ClassName.variable_name or self.variable_name.
- Changing a class variable affects all instances.

```
class Employee:
    company = "XYZ Corp"  # Class Variable
    def __init__(self, name, salary):
        self.name = name  # Instance Variable
        self.salary = salary  # Instance Variable
```

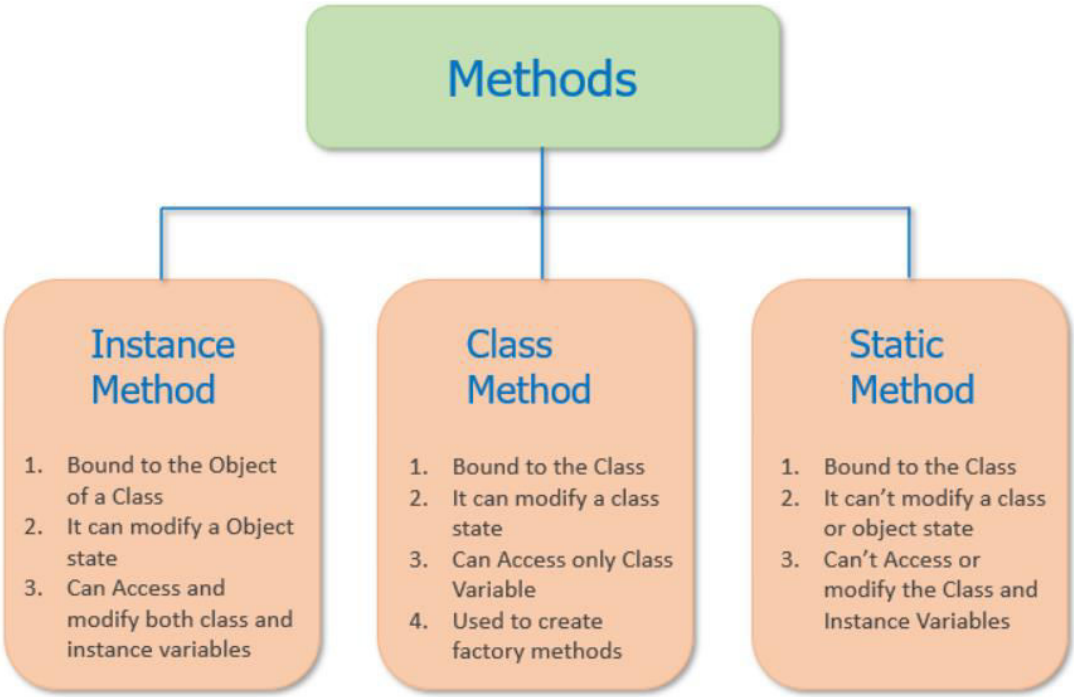
Key Differences

Feature	Instance Variable	Class Variable
Definition	Inside methods using `self.var`	Outside methods, shared across instances
Scope	Specific to an instance	Shared among all instances
Storage	Stored in `self.__dict__`	Stored in `class.__dict__`
Modification	Changes affect only that instance	Changes affect all instances if modified using `ClassName.var`



2. Methods in Python

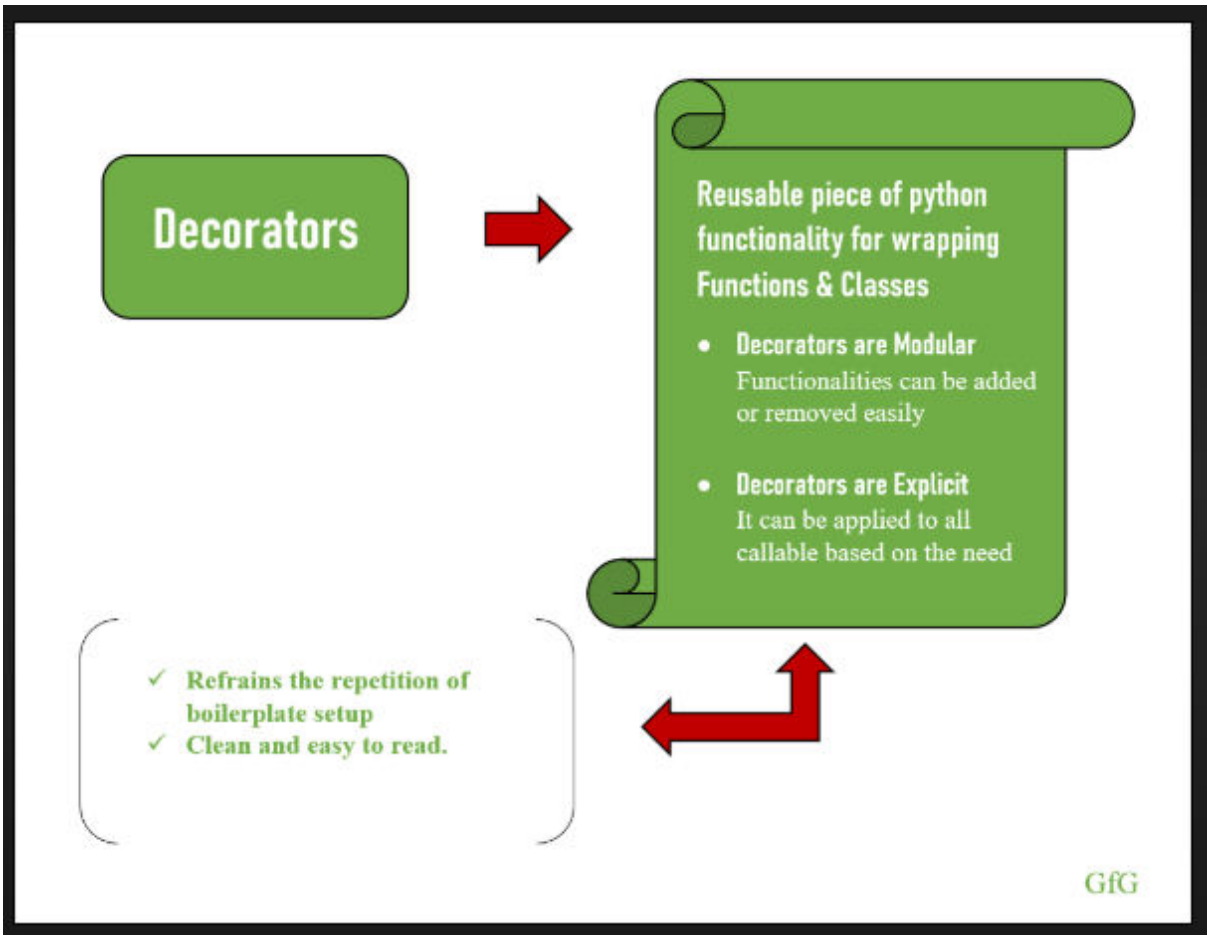
- **Instance Method**: Operates on an instance of the class.
- **Class Method**: Uses @classmethod decorator and operates on the class.
- **Static Method**: Uses @staticmethod decorator and does not operate on class or instance variables.



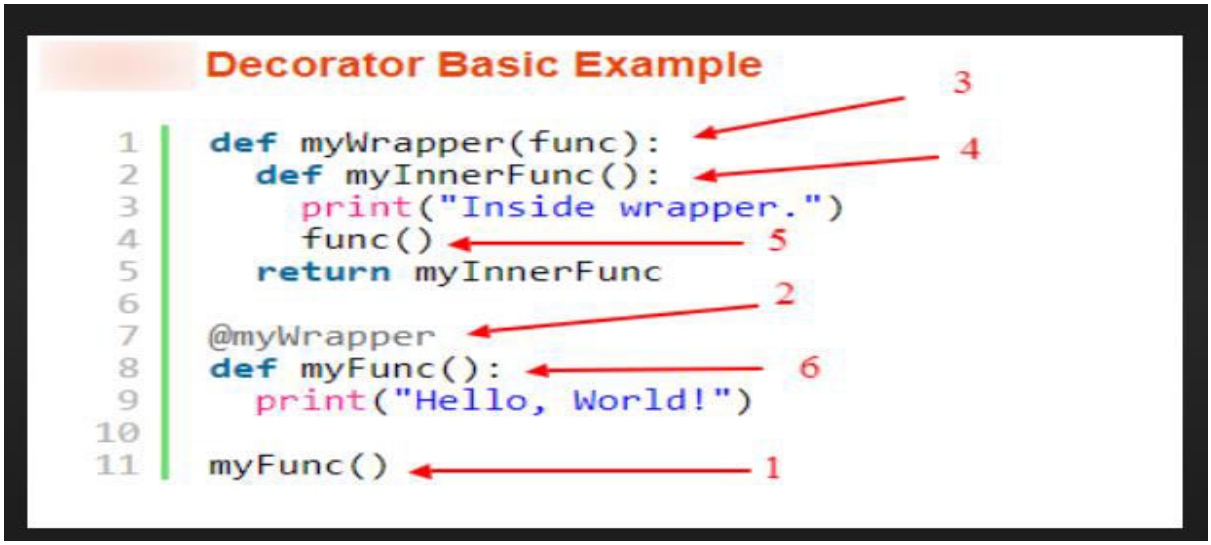
class method vs static method vs instance method

3. Python Decorators

In Python, decorators are a powerful and flexible way to modify or extend the behavior of functions or methods, without changing their actual code. A decorator is essentially a function that takes another function as an argument and returns a new function with enhanced functionality.



Decorators are often used in scenarios such as logging, authentication and memorization, allowing us to add additional functionality to existing functions or methods in a clean, reusable way.

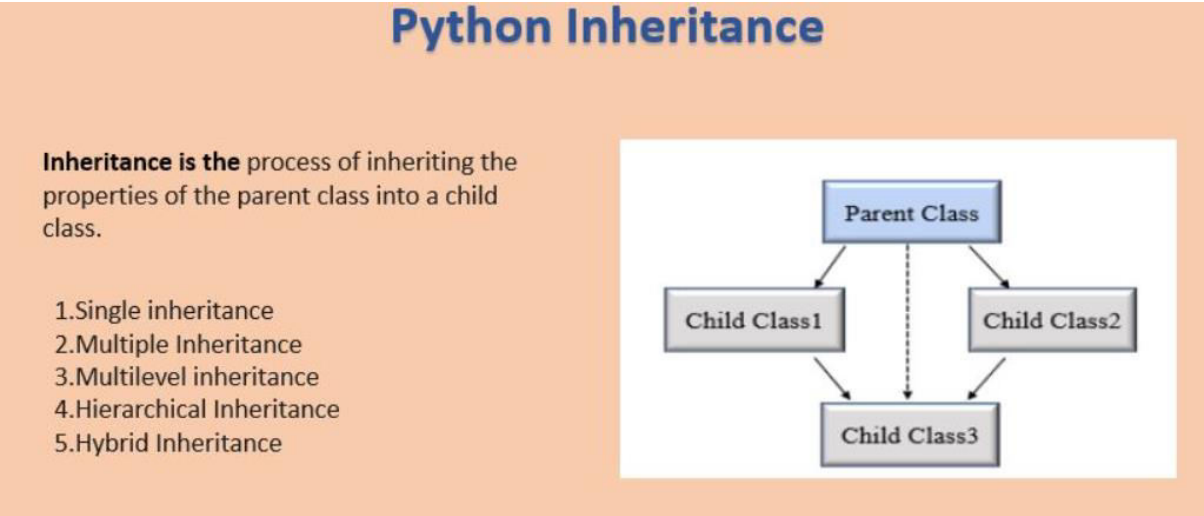


5. Inheritance in Python

Inheritance is a fundamental OOP concept where one class (child) derives attributes and methods from another class (parent). It promotes code reusability and allows hierarchical relationships between classes.

Why Use Inheritance?

- Avoids code duplication.
- Enhances code maintainability.
- Enables polymorphism (overriding methods in derived classes).
- Helps create modular and scalable code.



Types of Inheritance in Python

Inheritance allows one class to acquire the properties and methods of another class. Python supports different types of inheritance:

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

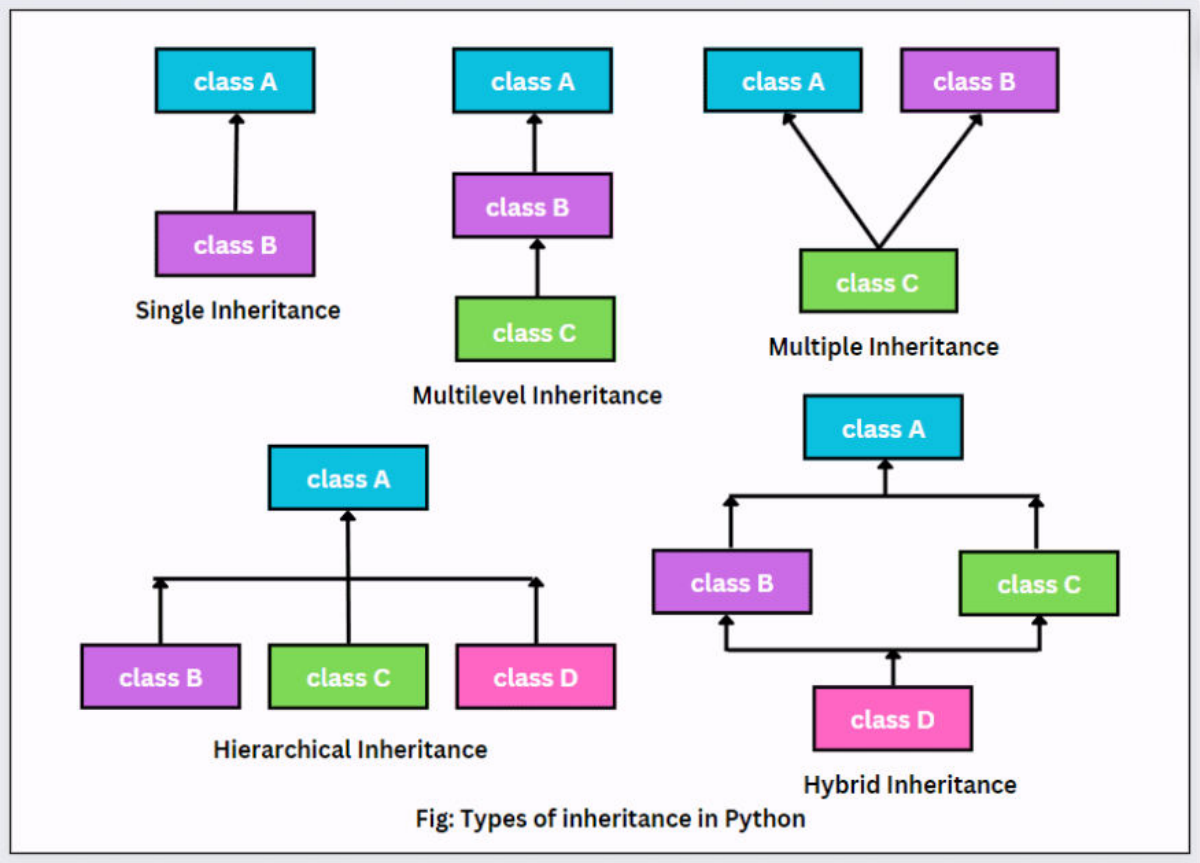


Fig: Types of inheritance in Python

1. Single Inheritance

One child class inherits from one parent class.

Key Point: The Child class has access to methods of the Parent class.

```python
class Parent:
    def display(self):
        print("This is the Parent class")

class Child(Parent):  # Inheriting Parent class
    def show(self):
        print("This is the Child class")

obj = Child()
obj.display()  # Inherited method
obj.show()     # Own method
```

2. Multiple Inheritance

A child class inherits from more than one parent class.

Key Challenge: If both parents have the same method, Python follows **Method Resolution Order (MRO)** to decide which one to execute.

Example:

```python
class Parent1:
    def method1(self):
        print("This is Parent1 class")

class Parent2:
    def method2(self):
        print("This is Parent2 class")

class Child(Parent1, Parent2):  # Inheriting from both Parent1 & Parent2
    def method3(self):
        print("This is Child class")

obj = Child()
obj.method1()
obj.method2()
obj.method3()
```

3. Multilevel Inheritance

A class inherits from a class, which in turn inherits from another class.

Example:

```python
class Grandparent:
    def grandparent_method(self):
        print("This is Grandparent class")

class Parent(Grandparent):
    def parent_method(self):
        print("This is Parent class")

class Child(Parent):
    def child_method(self):
        print("This is Child class")

obj = Child()
obj.grandparent_method()
obj.parent_method()
obj.child_method()
```

4. Hierarchical Inheritance

Multiple child classes inherit from a single parent class.

Example:

```python
class Parent:
    def parent_method(self):
        print("This is Parent class")

class Child1(Parent):
    def child1_method(self):
        print("This is Child1 class")

class Child2(Parent):
    def child2_method(self):
        print("This is Child2 class")

obj1 = Child1()
obj2 = Child2()

obj1.parent_method()  # Inherited from Parent
obj1.child1_method()  # Own method

obj2.parent_method()  # Inherited from Parent
obj2.child2_method()  # Own method
```

5. Hybrid Inheritance

A combination of two or more types of inheritance (e.g., multiple + multilevel).

Example:

```python
class A:
    def methodA(self):
        print("This is class A")

class B(A):  # Single Inheritance
    def methodB(self):
        print("This is class B")

class C:
    def methodC(self):
        print("This is class C")

class D(B, C):  # Multiple Inheritance
    def methodD(self):
        print("This is class D")

obj = D()
obj.methodA()  # Inherited from A
obj.methodB()  # Inherited from B
obj.methodC()  # Inherited from C
obj.methodD()  # Own method
```

Key Takeaways:

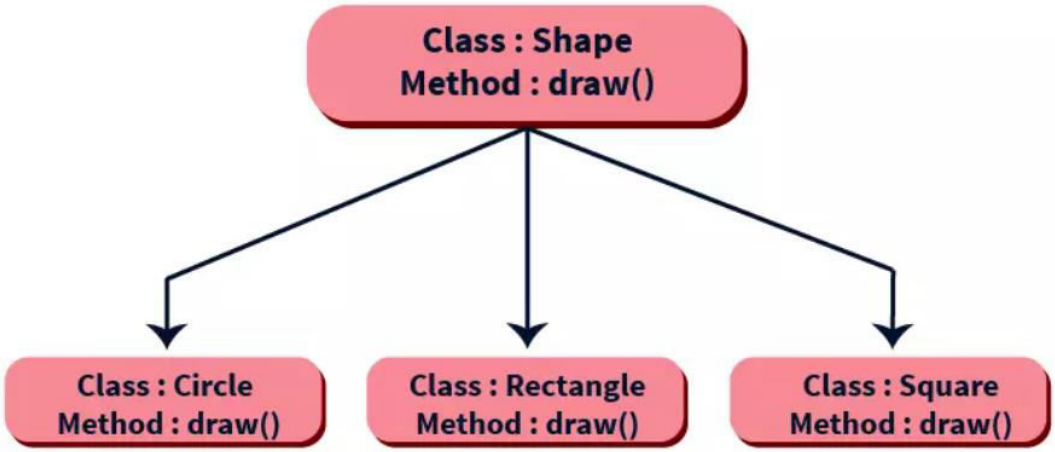
Type	Description
Single Inheritance	One child class inherits from one parent class
Multiple Inheritance	A child class inherits from multiple parent classes
Multilevel Inheritance	A class inherits from a class, which inherits from another class
Hierarchical Inheritance	Multiple child classes inherit from the same parent class
Hybrid Inheritance	A combination of multiple inheritance types

5. Polymorphism

What is Polymorphism?

Polymorphism is a key concept in Object-Oriented Programming (OOP) that allows the same function or method to be used with different data types or objects. In simple terms, it enables one interface to be used for different data types or classes.

Example: A single function can be used to process different types of objects, like a method draw() working for threeCircle , Rectangle and Square classes.



Why Use Polymorphism?

✅ Code Reusability – The same interface works for different objects.

✅ Flexibility – You can write more generic and scalable programs.

✅ Reduces Complexity – Avoids repetitive code and makes maintenance easier.

✅ Enhances Readability – Simplifies logic and improves organization.

Key Points About Polymorphism

✓ Polymorphism allows one method to be used for multiple object types.
✓ Supports method overloading, method overriding, operator overloading, and duck typing.
✓ Reduces redundancy, making programs more flexible and scalable.
✓ Helps in interface design, especially in frameworks and libraries.

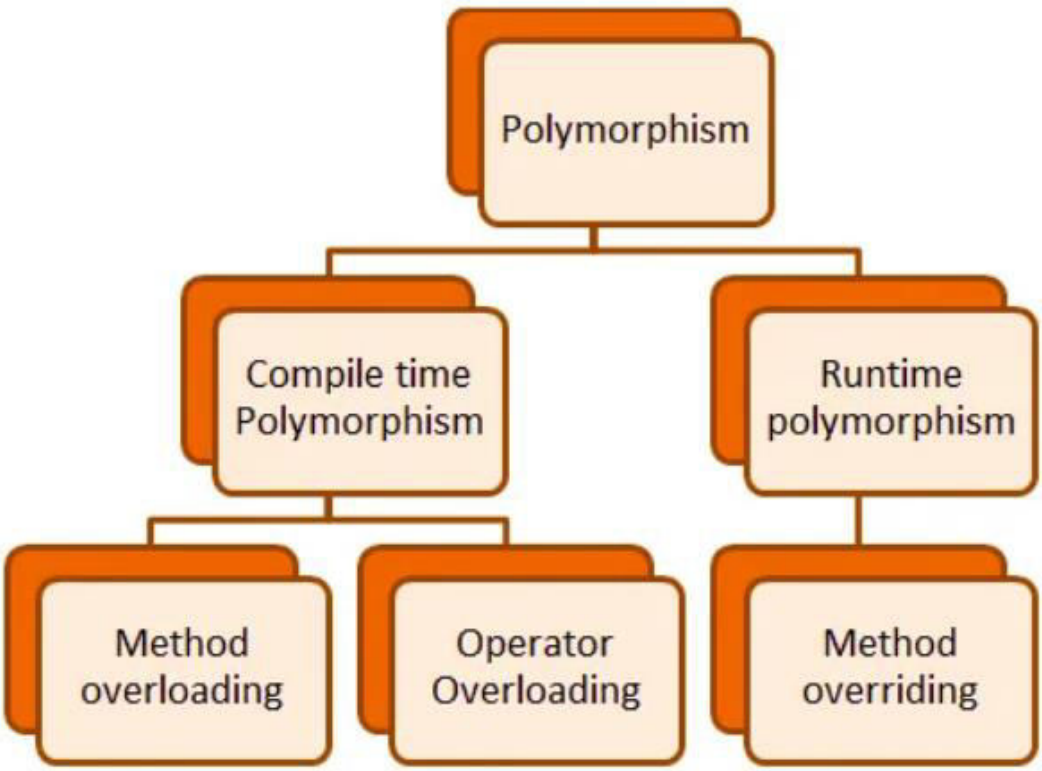
Advantages of Polymorphism

✅ Increases Code Efficiency – Same function for different data types.
✅ Improves Maintainability – Reduces code duplication.
✅ Enhances Extensibility – New classes can easily adapt existing methods.
✅ Facilitates Abstraction – Hides complex logic and presents a unified interface.

Limitations of Polymorphism

❌ Performance Overhead – More method calls can slow execution.
❌ Complex Debugging – Tracking overridden methods can be difficult.
❌ More Memory Usage – Dynamic binding may consume more memory.
❌ Not Always Necessary – Overuse can lead to unnecessary abstraction.

Types of Polymorphism in Python



1. Compile-time Polymorphism (Method Overloading)

Method overloading allows multiple methods in the same class to have the **same name but different parameters**. However, **Python does not support true method overloading** like Java or C++. Instead, we achieve it by using **default arguments or *args**.

Key Points:

- **Python does not support true method overloading.**
- **We use default arguments or *args to mimic overloading behavior.**

Example: Using Default Arguments

```python
class MathOperations:
    def add(self, a, b=0, c=0):
        return a + b + c

obj = MathOperations()
print(obj.add(5))          # Calls add(5, 0, 0) → Output: 5
print(obj.add(5, 10))      # Calls add(5, 10, 0) → Output: 15
print(obj.add(5, 10, 20)) # Calls add(5, 10, 20) → Output: 35
```

Here, the same `add()` method handles different numbers of arguments.

Example: Using *args for Overloading

```python
class MathOperations:
    def add(self, *args):
        return sum(args)

obj = MathOperations()
print(obj.add(5))          # Output: 5
print(obj.add(5, 10))      # Output: 15
print(obj.add(5, 10, 20)) # Output: 35
```

2. Runtime Polymorphism (Method Overriding)

Method overriding is a feature in **Object-Oriented Programming (OOP)** where a subclass provides a **new implementation** of a method that is

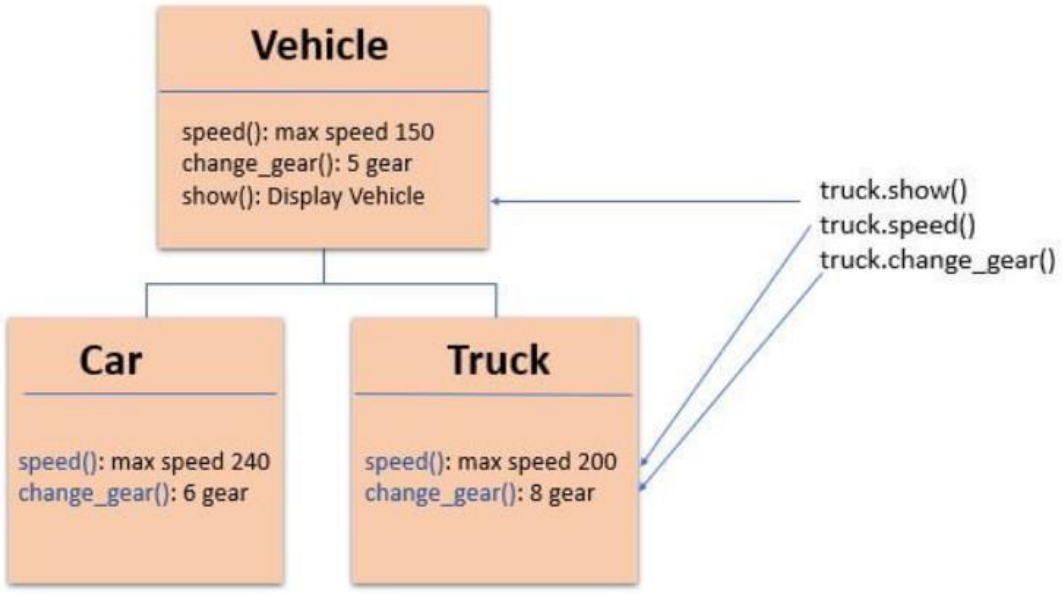
already defined in its **parent class**. This allows a subclass to **customize or extend** the behavior of a method without changing the parent class.

A child class overrides the method of its parent class.

Key Points:

✅ Method overriding occurs when a **child class** defines a method with the **same name and parameters** as a method in its **parent class**.
✅ The **child class method is called instead of the parent class method** when an instance of the child class is used.
✅ super() can be used to call the overridden method from the parent class.
✅ Used in **inheritance-based** programming to **modify or extend parent class behavior**.

Example of Method Overriding



Method overridden in Car and Truck class

Polymorphism with Inheritance

Example of Method Overriding

```python
class Parent:
    def show(self):
        print("This is the Parent class method")

class Child(Parent):
    def show(self):  # Overriding the method
        print("This is the Child class method")

# Creating objects
obj1 = Parent()
obj2 = Child()

# Calling methods
obj1.show()  # Calls Parent class method
obj2.show()  # Calls Child class method (overridden)
```

Output:

```kotlin
This is the Parent class method
This is the Child class method
```

Here, the `show()` method in the `Child` class overrides the method in the `Parent` class.

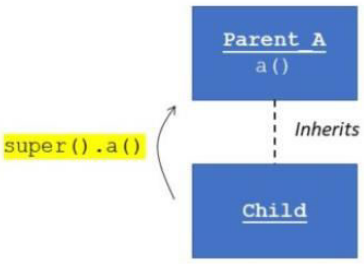
Using super() to Call Parent Method

Sometimes, we may want to keep the original functionality of the parent method while adding extra behavior in the child class. We can do this using the super() function.

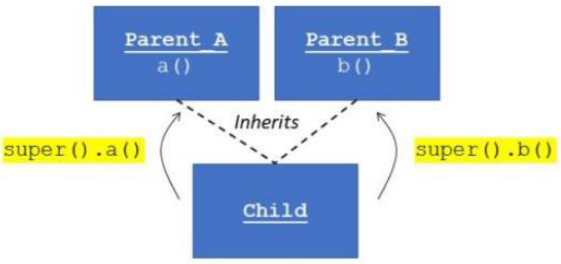
Python super()

Returns temporary object of the superclass to help you access its methods. Its purpose is to avoid using the base class name explicitly. It also enables ***multiple inheritance***.

A) Single Inheritance



B) Multiple Inheritance




```python
class Parent:
    def show(self):
        print("This is the Parent class method")

class Child(Parent):
    def show(self):
        super().show()  # Calls Parent class method
        print("This is the Child class method")

obj = Child()
obj.show()
```

Output:

```kotlin
This is the Parent class method
This is the Child class method
```

Here, `super().show()` calls the `show()` method from the **Parent** class before executing the additional statement in the **Child** class.

Method Overriding with __init__ (Constructor Overriding)

A child class can also override the __init__ constructor of the parent class.

```python
class Parent:
    def __init__(self, name):
        self.name = name
        print(f"Parent Constructor: Name is {self.name}")

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name)  # Call Parent constructor
        self.age = age
        print(f"Child Constructor: Age is {self.age}")

obj = Child("Alice", 25)
```

Output:

```pgsql
Parent Constructor: Name is Alice
Child Constructor: Age is 25
```

Here, the `Child` class overrides the `__init__` method of the `Parent` class but still calls it using `super().__init__(name)`.

Advantages of Method Overriding

✅ Encapsulation of Behavior: Allows child classes to modify parent class behavior without modifying the parent class.

✅ Code Reusability: Avoids duplicate code by extending parent class functionality.

✅ Flexible & Scalable Code: Allows easy customization for different use cases.

✅ Polymorphism: Enables dynamic method binding, where the method called depends on the object's type.

Limitations of Method Overriding

❌ Can Be Confusing: If not properly documented, it may be unclear which method is being executed.
❌ Performance Overhead: Calling super() adds an extra function call, which can slightly impact performance in performance-critical applications.
❌ Risk of Breaking Parent Class Functionality: If the child class doesn't call super(), it might miss important operations defined in the parent class.

3. Operator Overloading

Python allows operators like +, -, *, / to be **redefined** for custom classes using **magic methods**.

Key Points:

- Used for **custom behavior of operators** (+, -, *, etc.).
- Implemented using **magic methods** (__add__, __sub__, etc.).
- **Makes classes behave like built-in types**.

Example of Operator Overloading

```python
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __add__(self, other):
        return self.salary + other.salary  # Overloading '+'

emp1 = Employee("Alice", 50000)
emp2 = Employee("Bob", 60000)

print(emp1 + emp2)  # Output: 110000 (Overloaded '+')
```

Common Magic Methods for Operator Overloading

Operator	Magic Method
`+`	`__add__`
`-`	`__sub__`
`*`	`__mul__`
`/`	`__truediv__`
`%`	`__mod__`
`==`	`__eq__`
`<`	`__lt__`
`>`	`__gt__`

4. Duck Typing Polymorphism

Duck typing means **the type of an object is determined by its behavior rather than its class**.
"If it looks like a duck and quacks like a duck, it's a duck!"

Key Points:

- No need to check object type (isinstance is not required).
- Works as long as the object **has the required method**.
- **Encourages flexibility** in Python.

Example of Duck Typing

```python
class Bird:
    def sound(self):
        print("Chirp Chirp")

class Dog:
    def sound(self):
        print("Bark Bark")

def make_sound(animal):
    animal.sound()  # No need to check the type

bird = Bird()
dog = Dog()

make_sound(bird)  # Output: Chirp Chirp
make_sound(dog)   # Output: Bark Bark
```

Comparison of Polymorphism Types

Type	Description	Example
Method Overloading	Multiple methods with same name but different parameters (Python uses `*args` or default values)	`add(a, b)`, `add(a, b, c)`
Method Overriding	A subclass redefines a parent class method	`class Child(Parent): def show(self):`
Operator Overloading	Redefining how operators work on objects	`obj1 + obj2` calls `__add__()`
Duck Typing	Object's compatibility is based on behavior, not class	Any class with a `sound()` method can be passed to `make_sound(animal)`

Applications of Polymorphism

📌 Game Development – Different characters perform common actions like "attack" in different ways.

📌 GUI Applications – Buttons, text fields, and sliders implement click() differently.

📌 Machine Learning & AI – Models with the same train() method but different implementations.

📌 Web Frameworks – Functions like render() for different response types in Flask or Django.

📌 Automated Testing – Test cases that execute methods on multiple objects dynamically.

Where is Polymorphism Used?

🔷 In APIs & Frameworks – Django, Flask, PyTorch, TensorFlow, etc.

🔷 Database Management Systems (DBMS) – Same query structure but different implementations.

🔷 Software Development – Creating modular and scalable applications.

🔷 Networking Applications – Different communication protocols implementing the same interface.

Projects Based on Polymorphism

🚀 1. Shape Drawing Application – A program where draw() works for multiple shapes like circles, squares, etc.

🚀 2. AI Chatbot – Uses different response strategies (Text, Voice, Image).

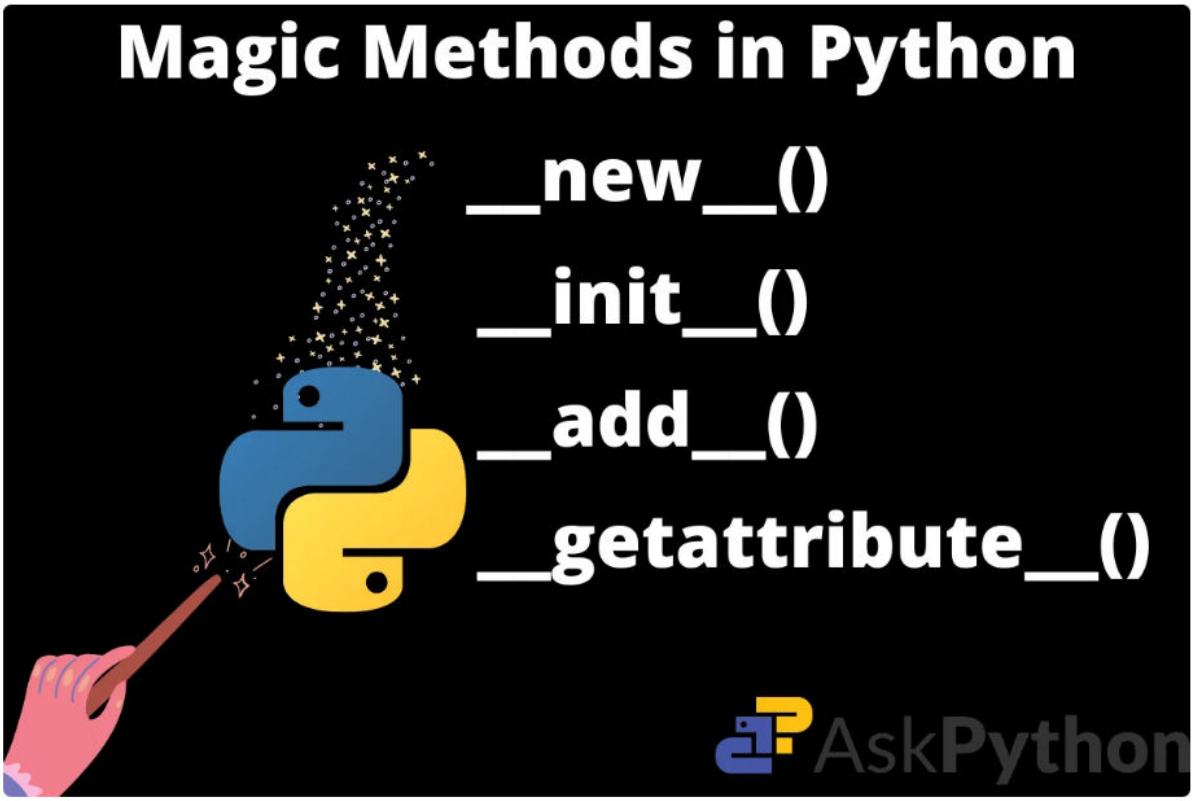
🚀 3. E-Commerce System – calculate_discount() for different customer types (Regular, VIP, etc.).

🚀 4. Smart Home Automation – turn_on() method working for lights, fans, ACs, etc.

🚀 5. Banking System – process_transaction() method handling deposits, withdrawals, transfers.

Magic Methods in Python

Magic methods (also called **dunder methods**, short for **double underscore methods**) are special methods in Python that begin and end with double underscores (e.g., __init__, __str__, __add__). They are used to define how objects of a class behave in specific situations, such as object creation, string representation, arithmetic operations, and more.



Why Use Magic Methods?

1. **Enhances Object-Oriented Programming (OOP):** Allows classes to define behaviors for built-in operations.
2. **Custom Behavior for Built-in Functions:** Enables custom implementations for functions like len(), str(), repr(), etc.
3. **Operator Overloading:** Allows objects to support operations like addition (+), comparison (>, <), etc.
4. **Readable and Maintainable Code:** Reduces boilerplate code and makes classes more Pythonic.
5. **Encapsulation of Functionality:** Handles internal operations cleanly within class definitions.

Key Magic Methods and Their Uses

Magic Method	Purpose
`__init__`	Constructor, initializes an object.
`__str__`	Defines human-readable string representation (`str()`).
`__repr__`	Defines developer-friendly string representation (`repr()`).
`__len__`	Defines behavior for `len()` .
`__getitem__` , `__setitem__`	Implements indexing and item setting.
`__call__`	Makes an object callable like a function.
`__add__` , `__sub__` , `__mul__`	Implements arithmetic operations.
`__eq__` , `__lt__` , `__gt__`	Implements comparison operations.
`__iter__` , `__next__`	Enables iteration in custom classes.

Types of Projects That Use Magic Methods

1. Data Science & Machine Learning:
    - Custom dataset classes (__getitem__, __len__).
    - Model objects with easy-to-read representations (__repr__).
2. Web Development:
    - Frameworks like Django and Flask use __str__, __repr__ for model classes.
    - Middleware classes often use __call__ for processing requests.
3. Game Development:
    - Operator overloading for game physics (__add__, __sub__ for vector math).
    - Iterable classes (__iter__) for handling game objects.
4. Finance & Banking Applications:
    - Custom number representations (__float__, __int__).

- Operator overloading for financial calculations.

5. Automation & Scripting:
   - __enter__ and __exit__ for context managers in resource management.

Example

```python
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __str__(self):
        return f"Employee: {self.name}, Salary: {self.salary}"

    def __add__(self, other):
        return self.salary + other.salary

# Creating objects
emp1 = Employee("Alice", 50000)
emp2 = Employee("Bob", 60000)

print(emp1)  # Calls __str__
print(emp1 + emp2)  # Calls __add__
```

Output:

```yaml
Employee: Alice, Salary: 50000
110000
```

Here, `__str__` makes printing readable, and `__add__` enables salary addition using `+`.

Advantages of Magic Methods

✅ Improves Code Readability:

- Example: str(obj) calls __str__, making objects more understandable.

✅ Reduces Boilerplate Code:

- Example: Instead of writing a separate function for addition, use __add__ directly.

✅ Enables Operator Overloading:

- Example: a + b can work for custom objects if __add__ is defined.

✅ Better Debugging with __repr__:

- Helps developers inspect objects clearly in logs.

✅ Essential for Custom Data Structures:

- Enables Pythonic handling of lists, dictionaries, and other collections.

Limitations of Magic Methods

❌ Can Make Code Hard to Debug:

- If overused, it may introduce unexpected behaviors.

❌ Performance Overhead:

- Custom implementations of operators may be slower than built-in types.

❌ Less Explicit than Normal Methods:

- A function like add_numbers(a, b) is more readable than a + b if a and b are non-standard objects.

❌ Not Always Intuitive:

- Some magic methods (like __getattr__, __getattribute__) can introduce complex logic that is hard to follow.

Abstract Class & Abstract Method

Abstract Class & Abstract Method in Python

Abstract Class

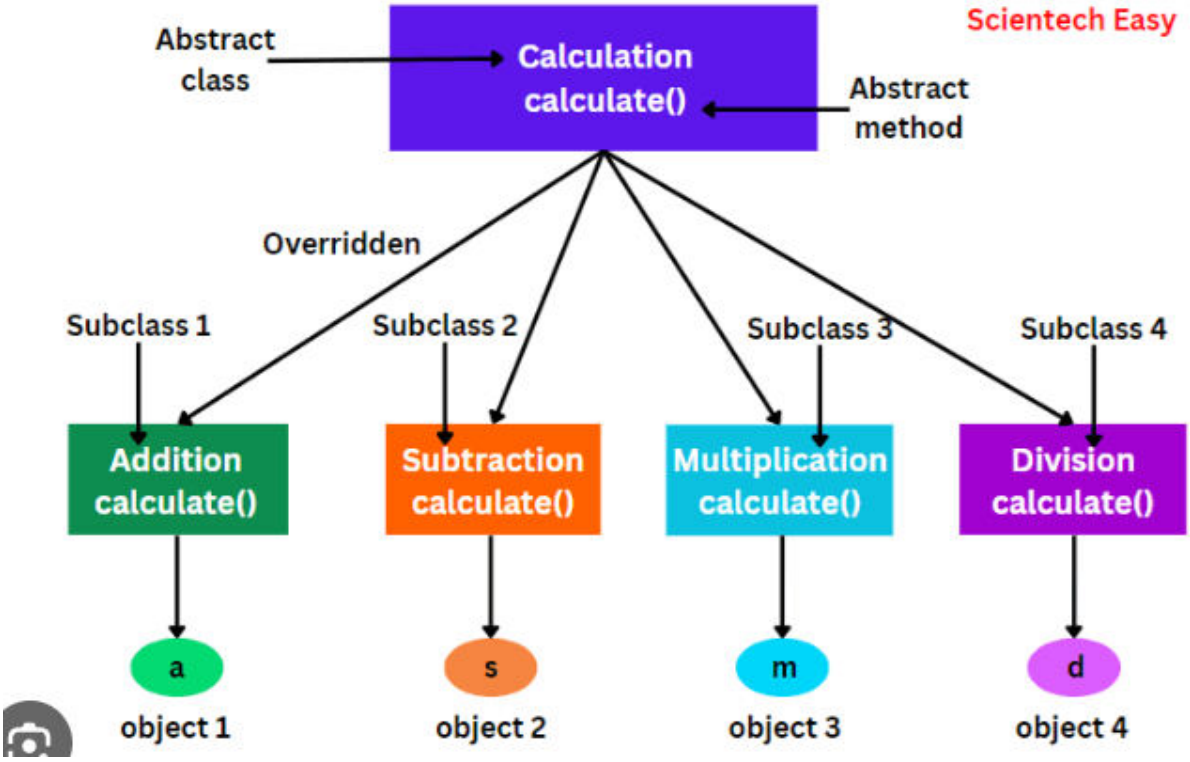
An abstract class is a class that cannot be instantiated directly. It serves as a blueprint for other classes and often includes one or more abstract methods. Abstract classes are used to define common behavior that must be implemented by subclasses.

Abstract Method

An abstract method is a method that is declared in an abstract class but does not have any implementation. Subclasses that inherit from the abstract class must provide their own implementation of the abstract methods.

Difference Between Abstract Class & Abstract Method

Feature	Abstract Class	Abstract Method
Definition	A class that contains at least one abstract method and cannot be instantiated	A method that is declared but has no implementation in the base class
Implementation	Can have both abstract and concrete methods	Must be implemented by subclasses
Instantiation	Cannot be instantiated directly	Implemented in child class
Purpose	Provides a common structure for subclasses	Enforces method implementation in derived classes



Why Use Abstract Classes & Methods?

1. Encapsulation of Common Behavior – Abstract classes allow defining common behavior that all subclasses should have.
2. Code Reusability – Reduces duplicate code by implementing shared logic in the abstract class.
3. Enforces a Contract – Ensures that all subclasses implement the required methods.

4. Supports Polymorphism – Abstract classes allow subclasses to have different implementations while maintaining a common interface.

Where to Use Abstract Classes & Methods?

- Framework Design – When creating a base class for multiple related subclasses.
- Plugin Architecture – Used to enforce methods in plugins without defining exact behavior.
- Game Development – A base class for different types of game objects (e.g., Character, Enemy).
- Machine Learning Pipelines – Defining preprocessing steps that all models should follow.
- E-commerce Applications – Abstracting payment processing for different payment gateways.

Advantages of Abstract Classes & Methods

✅ Encourages Code Reusability – Avoids duplication of code by defining common logic.
✅ Ensures Consistency – Enforces a structure across multiple subclasses.
✅ Supports Inheritance & Polymorphism – Allows dynamic method overriding.
✅ Improves Maintainability – Helps in managing and extending large codebases easily.

Limitations of Abstract Classes & Methods

❌ Cannot Instantiate – Abstract classes cannot be used directly.

❌ Less Flexibility than Interfaces – Python does not have interfaces, but in other languages, interfaces provide more flexibility than abstract classes.

❌ Increases Complexity – Requires additional classes to define and extend functionality.

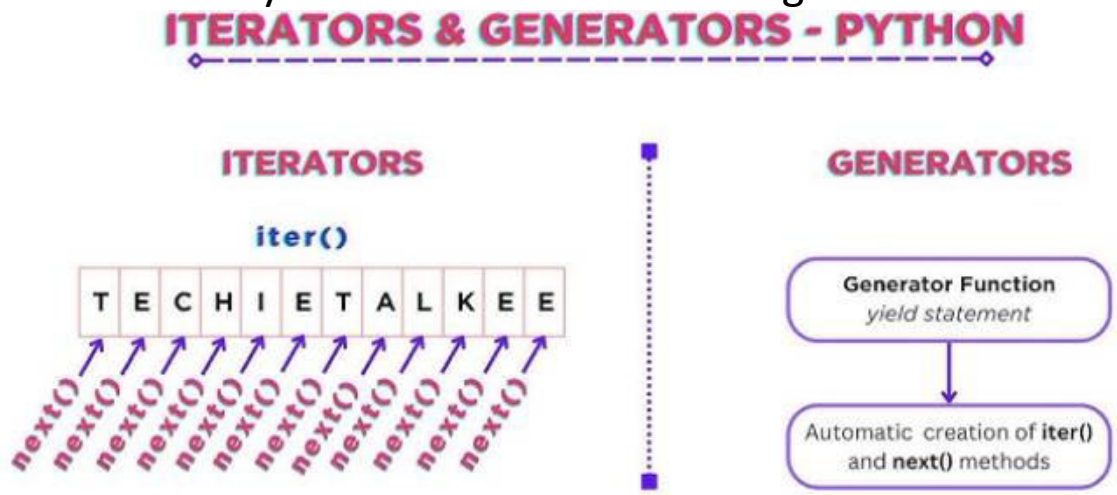
Iterators & Generators

What are Iterators & Generators?

- **Iterators:**

  An **iterator** is an object that allows traversal through all the elements of a collection (like lists, tuples, dictionaries, etc.) **one at a time**. It implements the __iter__() and __next__() methods.Implements __iter__() and __next__().

- **Generators**
  A **generator** is a simpler way to create iterators using the yield keyword. Unlike iterators, generators do not store all values in memory; instead, they generate values **on the fly** when needed.Uses yield for on-demand value generation.



Types of Iterators & Generators

Iterators

1. **Built-in Iterators:** Strings, lists, tuples, dictionaries, and sets are iterable and can be used with loops.
2. **User-defined Iterators:** Custom iterators created using classes by implementing __iter__() and __next__() methods.

Generators

1. **Generator Functions:** Created using the yield keyword inside a function.
2. **Generator Expressions:** Similar to list comprehensions but use parentheses ( ) instead of square brackets [ ].

Feature	Iterators	Generators
Definition	Object that implements `__iter__()` and `__next__()`	Function that uses `yield` to produce values lazily
Memory Usage	Stores all values in memory	Generates values on demand (efficient memory usage)
Implementation	Created using classes & methods (`__iter__()`, `__next__()`)	Created using functions with `yield`
Reusability	Can be reused by resetting manually	Cannot be restarted automatically
Performance	Uses more memory if dataset is large	More efficient for large datasets

Why Use Iterators & Generators?

✅ Efficient memory usage (no need to store entire data in memory)

✅ Faster execution for large datasets

✅ Useful for streaming large data (files, real-time data

processing)

✅ Lazy evaluation (only computes the next value when needed)

Applications of Iterators & Generators

- Big data processing (handling large files, logs, or databases)
- Web scraping (fetching data from multiple pages efficiently)
- Machine learning & AI (processing datasets in chunks)
- Streaming services (loading data dynamically, e.g., YouTube, Spotify)

Where to Use Iterators & Generators?

✅ When working with large datasets (to avoid memory overload)

✅ When you don't need to store all values at once

✅ When handling real-time data or infinite sequences

Projects Using Iterators & Generators

🔷 Log file reader – Process large logs efficiently using generators

🔷 Live stock market tracker – Use iterators to fetch real-time data

🔷 Database row fetcher – Retrieve large query results using lazy loading

🔷 API pagination handler – Fetch paginated data from web APIs

Key Points

- Iterators use __iter__() and __next__() methods.
- Generators use yield and provide lazy evaluation.
- Generators save memory and improve performance.
- Iterators can be reset, but generators cannot.
- Ideal for big data processing, AI, and streaming services.

Advantages & Limitations

✅ Advantages

✓ Memory efficient – Generates values on demand instead of storing them.
✓ Faster execution – No need to create a full list before iteration.
✓ Simpler syntax – Generators reduce the need for manual class-based iterators.
✓ Scalable – Useful for processing millions of records without memory overflow.

❌ Limitations

❌ Generators cannot be restarted – Once exhausted, they cannot be reused.
❌ Debugging is harder since values are generated on the fly.
❌ Slightly slower execution for small datasets compared to lists.

Applications of OOP

- Game Development
- GUI Applications
- Database Management
- Machine Learning & AI
- Web Development

Projects Based on OOP

- **Library Management System**
- **ATM Simulation**
- **E-commerce System**
- **Student Management System**
- **Bank Account System**

Interview Questions & Answers

Basic Level

1. **What is a class and an object?**

   Ans :A **class** is a blueprint for creating objects, defining properties (variables) and behaviors (methods).
   ⍰ An**object** is an instance of a class with its own values.

2. **What is self in Python?**

Ans:self represents the **current instance** of the class and allows access to its attributes and methods.It must be explicitly used in instance methods.

3. **What are instance variables and class variables?**

Feature	Instance Variable	Class Variable
Defined	Inside a method using `self.variable_name`	Outside methods, shared across instances
Scope	Specific to an instance	Shared among all instances
Modification	Affects only that instance	Affects all instances if modified via class

4. **Explain the purpose of the __init__ method in Python.**

Ans:The __init__ method is a constructor that initializes instance variables when an object is created. It is automatically called when an instance is created.

5. **What is the difference between a function and a method in Python?**

Feature	Function	Method
Defined	Outside a class	Inside a class
Belongs to	No specific object	A specific object or class
Called by	Name	`object.method()`

6. **What are the four main principles of OOP?**

Encapsulation: Wrapping data and methods together.

Abstraction: Hiding implementation details and showing only the necessary features.

Inheritance: One class acquires properties of another.

Polymorphism: Methods or functions behaving differently based on input or class.

Intermediate Level

1. **What is the difference between @classmethod and @staticmethod?**
   - @classmethod operates on the class, while @staticmethod does not require instance or class reference.

Decorator	`@classmethod`	`@staticmethod`
Works with	Class	Neither instance nor class
First parameter	`cls` (class reference)	No implicit first argument
Use case	Modify class variables	Utility functions

2. **What is method overloading?**

Ans: Python does not support method overloading directly, but it can be achieved using default arguments.

3. **What is method overriding in Python?**
4. **How does inheritance work in Python? Give an example**

   Ans: Inheritance allows a class to inherit attributes and methods from another class.

5. **What is multiple inheritance, and what are its challenges?**

   Ans :**Multiple inheritance** allows a class to inherit from multiple parent classes.

Challenges:

- **Diamond Problem:** Confusion about which method to execute in case of ambiguity.

Advanced Level

1. What is the difference between an abstract class and an interface?

- Abstract classes can have both concrete and abstract methods, while interfaces (in languages like Java) contain only abstract methods.

Feature	Abstract Class	Interface (in Java)
Methods	Can have abstract & concrete methods	Only abstract methods
Implementation	Can provide some implementation	No implementation
Instantiation	Cannot be instantiated	Cannot be instantiated

Example:

```python
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass
```

2. What are magic methods in Python?

Ans: Special methods with double underscores (__) that allow operator overloading.

3. Explain method resolution order (MRO) in Python.

Ans:MRO determines the order in which base classes are searched when executing a method.

MRO determines the order in which methods are inherited when multiple parents exist.

Python follows **C3 Linearization (Depth-First, Left-to-Right)**.

4. What is abstraction? How is it implemented in Python?

Ans: Abstraction hides unnecessary details from the user.Implemented using **abstract classes** (ABC module).

Example:

```python
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass

print(D.mro())  # Shows method resolution order
```

5. How does multiple inheritance handle conflicts?

Ans: Python resolves conflicts using **Method Resolution Order (MRO)** based on the **C3 Linearization Algorithm**.

6. What is a destructor in Python?

Ans:A destructor (__del__ method) is called when an object is deleted or goes out of scope to free up resources.

7. What is the difference between procedural and object-oriented programming?

Ans:

Feature	Procedural Programming	Object-Oriented Programming
Approach	Function-based	Object-based
Code Reusability	Low	High (via inheritance)
Data Handling	Data is exposed	Data is encapsulated